

1

Consider the following SQL schema for a university course registration system:

```
Enrollment(  
  studentID INTEGER, -- 4 bytes  
  courseCode CHAR(8), -- 8 bytes  
  semester CHAR(6), -- 6 bytes  
  grade FLOAT, -- 8 bytes  
  credits SMALLINT -- 2 bytes  
)
```

Assume:

- No null values are allowed.
- Records are stored in fixed-length format (no variable-length fields).
- Each disk page is 1024 bytes.
- No spanning is allowed: a record cannot be split across two pages.

i

$4 + 8 + 6 + 8 + 2 = 28$ bytes per Enrollment record

ii

$\lfloor 1024/28 \rfloor = \lfloor 36.5714285714 \rfloor = 36$ records per page.

iii

Since it is given that a record cannot be split across two pages:

$\lceil 42,000/36 \rceil = \lceil 1166.6666666667 \rceil = 1167$ total pages for all the records

iv

If the page size is doubled then the number of pages required will more than half. This is because now we would be able to fit more than double the amount of records per page. This is shown as:

$$\lfloor 2048/28 \rfloor = \lfloor 73.1428571429 \rfloor = 73 \text{ records per page, which is one more than simply doubling}$$

Thus we can recalculate the number of pages required as:

$$\lceil 42,000/73 \rceil = \lceil 575.3424657534 \rceil = 576 \text{ which is less than half of the previous required pages.}$$

This results in a change of 0.4935732648 which one could argue is roughly half but not exactly half.

2

A slotted page has the following layout:

- Page header: stores (a) the total number of records and (b) the offset to the start of free space. Header size = 20 bytes.
- Slot directory: grows upward from the end of the header. Each slot entry = 4 bytes and stores the byte offset of its corresponding record within the page.
- Records grow downward from the top of the page into free space.
- Page size = 1024 bytes. Each Enrollment record from Question 1 is stored here.

i

Record size is the same from problem 1 so 28 bytes. Slots are 4 bytes each and the header is 20 bytes.

Thus the total amount of pages stored in a single page is:

$$\lfloor (1024 - 20)/(28 + 4) \rfloor = \lfloor 31.375 \rfloor = 31 \text{ records per page including headers and slot information.}$$

ii

a

The bytes consumed by the records is $28 * 8 = 224$ bytes.

b

The header is still 20 bytes because this is constant

c

The bytes for the slots is $8 * 4 = 32$

d

Therefore the remaining space is $1024 - 224 - 20 - 32 = 748$ bytes

iii

According to the structure given, the header is at the bottom of our space and the slot directory grows upward from above the header and the actual records start at the top and grow down.

We can start by viewing where free space starts (where slots end) in the header and go to that location. From there we read the location of where the last slot points (where the last record ends). Then we go there and append the record onto the end.

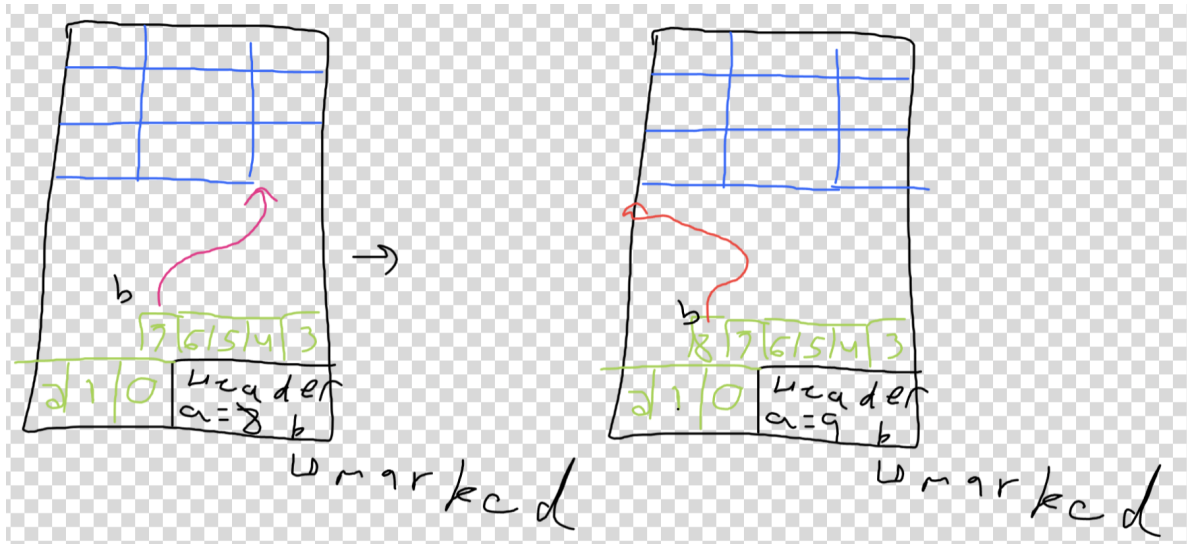


Figure 1: Image

iv

The simple / lazy of the deletion is simply marking the 4th slot (which is at index 3 because we are 0 based indexing) by setting the offset to -1 which would be invalid. This is essentially deleting the data without 0ing out the record data.

As shown in the image below, the 4th slot pointer points to the correct location corresponding to the 4th record but after the deletion it points to an invalid location

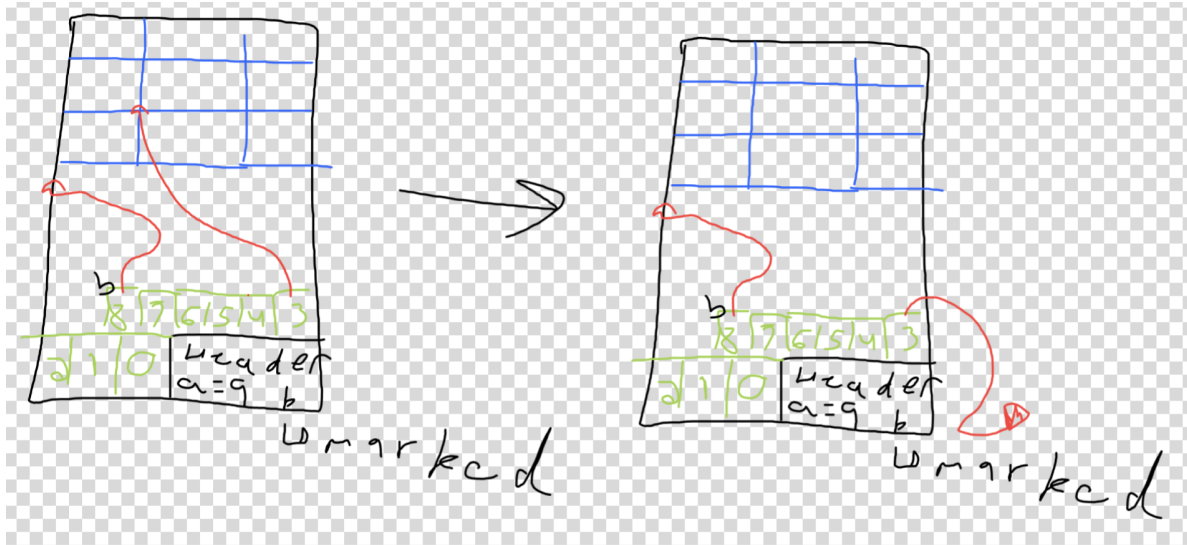


Figure 2: Image

3

i

We simply perform the mod 3 on each to determine which server holds them.

$H(\text{book})$

So book is server 2

$H(\text{patron})$

So patron is server 2

$H(\text{loan})$

So loan is server 0

ii

This problem does not have sufficient information to fill out the tables. This is because it suggests the existence of 2 different books that are loaned under Alice Chen but only one book's information is given (and only the loan information for this book). As such, I will do the best given what I have.

Books

book_id	title	author	year	copies
ISBN001	Database Internals	Alex Petrov	2019	5

Patrons

patron_id	name	email
P2201	Alice Chen	alice@uni.edu

Loans

loan_id	patron_id	book_id	due_date	returned
L9901	P2201	ISBN001	2025-05-01	false

iii

A scenario in which the JSON document store design is preferable is for when we are trying to lookup what books are loaned under a patrons account. This is because that information is found by looking up the id of the patron in the document store, then under the “loans” property is the list of all the books under that patron’s account. On the other hand, this same query in the relational schema would require an iteration over every loan in the Loans table looking for matches of the patron_id. Thus the document store would be an $O(1)$ lookup while the relational schema would be $O(n)$ for this query.

If we are trying to find the names of people that haven’t returned their books, then the relational schema is better. This is because joins are more taxing on JSON style stores and additionally our Patrons table in the relational schema has one less attribute/property than the JSON store. This means that we would be reading less bytes from this attribute in our relational schema compared to the JSON store. Thus for these join queries, particularly ones that join on the patrons table, are performed better in the relational schema.

4

i

C. They store each column’s values together, enabling efficient compression and column scans.

ii

B. Stored together as a contiguous block, often in compressed form

iii

C. `SELECT AVG(grade), COUNT(*) FROM Enrollment WHERE semester = 'FA25';`

iv

Since it is given that each field is 8 bytes, the column oriented store only reads the amount field for all entries, thus reading $8 \times 2,000,000 = 16,000,000$ bytes.

On the other hand the row oriented approach needs to read every field for each entry. Since each entry has 4 fields and is 8 bytes, the total bytes read from disk is $32 \times 2,000,000 = 64,000,000$ bytes.